

Task1: To Implement how to List Active Jobs, Display Status Information, Retrieve Job IDs, Format Output, Update State Information, Test Job Listings

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
```

```
#define MAX_JOBS 10
```

```
typedef struct {
    int job_id;
    pid_t pid;
    int active;
} Job;
```

```
Job jobs[MAX_JOBS];
int job_count = 0;
```

```
void add_job(pid_t pid) {
    if (job_count >= MAX_JOBS)
        return;

    jobs[job_count].job_id = job_count + 1;
    jobs[job_count].pid = pid;
    jobs[job_count].active = 1;

    job_count++;
}
```

```
void update_status() {
    for (int i = 0; i < job_count; i++) {
        if (!jobs[i].active)
            continue;
```

```
        int status;
        pid_t result = waitpid(
            jobs[i].pid,
            &status,
            WNOHANG
        );
```

```
        if (result == jobs[i].pid) {
            jobs[i].active = 0;
        }
    }
}
```

```
void list_jobs() {
    printf("\n%-10s %-10s %-15s\n",
        "Job ID", "PID", "Status");

    printf("-----\n");

    for (int i = 0; i < job_count; i++) {
```

```

void list_jobs() {
    printf("\n%-10s %-10s %-15s\n",
           "Job ID", "PID", "Status");

    printf("-----\n");

    for (int i = 0; i < job_count; i++) {
        printf("%-10d %-10d %-15s\n",
               jobs[i].job_id,
               jobs[i].pid,
               jobs[i].active ? "RUNNING" : "COMPLETED");
    }
}

int main() {
    for (int i = 0; i < 3; i++) {
        pid_t pid = fork();

        if (pid < 0) {
            perror("fork");
            return 1;
        }
    }
}

```

```

        if (pid == 0) {
            sleep(5 + i * 2);
            exit(0);
        }

        add_job(pid);
    }

    printf("Active jobs created.\n");

    for (int i = 0; i < 4; i++) {
        update_status();
        list_jobs();
        sleep(3);
    }

    for (int i = 0; i < job_count; i++) {
        if (jobs[i].active) {
            waitpid(jobs[i].pid, NULL, 0);
            jobs[i].active = 0;
        }
    }

    printf("\nFinal job status:\n");
    list_jobs();
}

```

```
shreyas@Grigori:~$ mkdir -p ~/job_listing
cd ~/job_listing
shreyas@Grigori:~/job_listing$ nano task1.c
shreyas@Grigori:~/job_listing$ gcc task1.c -o task1
shreyas@Grigori:~/job_listing$ ./task1
Active jobs created.
```

Job ID	PID	Status
1	976	RUNNING
2	977	RUNNING
3	978	RUNNING

Job ID	PID	Status
1	976	RUNNING
2	977	RUNNING
3	978	RUNNING

Job ID	PID	Status
1	976	COMPLETED
2	977	RUNNING
3	978	RUNNING

Job ID	PID	Status
1	976	COMPLETED
2	977	COMPLETED
3	978	COMPLETED

```
shreyas@Grigori:~$ ps -o pid,ppid,state,cmd
  PID   PPID  S  CMD
   986   980  S  -bash
  1002   986  R  ps -o pid,ppid,state,cmd
shreyas@Grigori:~$ ./task1
Enter a command (type 'exit' to quit): pwd
You entered: pwd
Enter a command (type 'exit' to quit): ls
You entered: ls
Enter a command (type 'exit' to quit): whoami
You entered: whoami
Enter a command (type 'exit' to quit): exit
Exiting...
shreyas@Grigori:~$ |
```

Task2: To design how to Identify Target Job, Transfer Terminal Control, Resume Processes, Wait for Completion, Update Job State, Test Foreground Switching

```
GNU nano 7.2 task2.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <signal.h>
#include <termios.h>

int main() {
    pid_t pid;
    int status;

    pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        setpgid(0, 0);

        printf("Foreground job started.\n");
        printf("PID: %d\n", getpid());
        printf("PGID: %d\n", getpgrp());
    }
}
```

```

        sleep(5);

        printf("Foreground job completed.\n");

        exit(0);
    }

    setpgid(pid, pid);

    printf("Target job PID: %d\n", pid);
    printf("Transferring terminal control...\n");

    tcsetpgrp(STDIN_FILENO, pid);

    waitpid(pid, &status, 0);

    tcsetpgrp(STDIN_FILENO, getpgrp());

    if (WIFEXITED(status)) {
        printf("Job completed with exit status: %d\n",
            WEXITSTATUS(status));
    }

    printf("Terminal control returned to parent.\n");

    return 0;

```

```

shreyas@Grigori:~/job_listing$ nano task2.c
shreyas@Grigori:~/job_listing$ gcc task2.c -o task2
shreyas@Grigori:~/job_listing$ ./task2
Target job PID: 1012
Transferring terminal control...
Foreground job started.
PID: 1012
PGID: 1012
Foreground job completed.

[1]+  Stopped                  ./task2
shreyas@Grigori:~/job_listing$ ./task2
Target job PID: 1016
Transferring terminal control...
Foreground job started.
PID: 1016
PGID: 1016
Foreground job completed.

[2]+  Stopped                  ./task2
shreyas@Grigori:~/job_listing$ ps -o pid,ppid,pgid,state,cmd
  PID   PPID   PGID S  CMD
   937    932    937 S  -bash
  1011    937   1011 T  ./task2
  1015    937   1015 T  ./task2
  1017    937   1017 R  ps -o pid,ppid,pgid,state,cmd
shreyas@Grigori:~/job_listing$ rm -f task2
shreyas@Grigori:~/job_listing$

```

```
shreyas@Grigori:~$ ps -o pid,ppid,pgid,tpgid,state,cmd
  PID    PPID    PGID    TPGID S  CMD
   986    980    986    1013 S  -bash
  1013    986    1013    1013 R  ps -o pid,ppid,pgid,tpgid,state,cmd
shreyas@Grigori:~$ ./task2
Process started
PID : 1014
PPID : 986
Process State: Running
Process State: Sleeping/Waiting
Process State: Running again
Process State: Terminating
shreyas@Grigori:~$ |
```